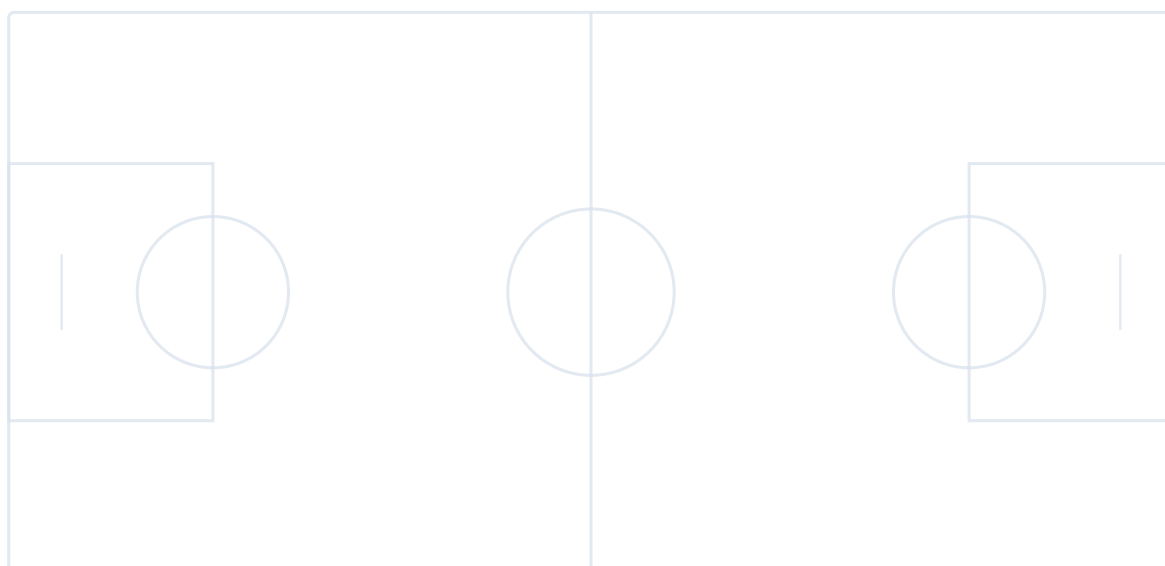

Python「黑客松」實戰分析

籃球運動數據

Python • Computer Vision • Sports Analytics



Contents

課程導論 | 籃球運動數據分析的核心問題

iv

1 從影像像素到真實球場：座標、標註與 Homography	1
1.1 先建立直覺：相機看到的不是「距離」，而是「投影」	1
1.2 資料標註：不是「框越精細越好」，而是要符合問題	1
1.2.1 四種常見標註及其代價	2
1.2.2 標註規則比標註工具更重要	2
1.3 Homography：把一個平面映射到另一個平面	2
1.3.1 它能做什麼，也不能做什麼	2
1.3.2 為什麼要用齊次座標？	3
1.3.3 DLT、RANSAC 與重投影誤差	3
1.3.4 標定點的選擇原則與品質檢查	3
1.4 從 BBOX 到球場位置：為什麼常取腳底中心？	4
1.5 工具選擇與實務流程	5
1.6 本日研究檢查表	5
2 模型如何找到球員與籃球：Object Detection 與 BBOX-to-BEV	6
2.1 先說需求：為什麼不能只做影像分類？	6
2.2 偵測技術的演進：從滑動視窗到端到端模型	6
2.3 現代 Detector 的三個主要部件	7
2.3.1 為什麼籃球比球員難偵測？	7
2.4 Confidence、Precision、Recall：三個常被誤解的概念	7
2.4.1 Confidence 門檻是一個取捨旋鈕	7
2.4.2 IoU：框與框重疊多少	8
2.5 NMS：為什麼同一個人會先出現很多候選框？	8
2.6 從 Detection 到 BEV：資料鏈與誤差鏈	8
2.7 常見工具與適用情境	9
2.8 Detection 結果的解讀與品質評估	9
3 從每一幀的框，到持續身分與隊伍：Multi-Object Tracking	10
3.1 什麼是追蹤？需求不是「畫軌跡線」而已	10

3.2	追蹤技術的演進：從光流到資料關聯	10
3.3	Tracking-by-Detection 的完整流程	11
3.3.1	Motion model：先猜它會去哪裡	11
3.3.2	Cost matrix：每一格都是一個「可能是同一人嗎？」	11
3.3.3	Hungarian assignment：不是每個 track 自己搶最近的框	11
3.4	ByteTrack：為什麼低分框有時很有價值？	11
3.5	Track lifecycle：身分是有生命週期的	12
3.6	ReID 與隊伍辨識：位置不夠時，加入外觀	13
3.6.1	Re-identification 是什麼？	13
3.6.2	無標籤隊伍分群	13
3.7	Tracking-to-BEV：把時間與空間合併	13
3.8	追蹤應該怎麼評估？	14
4	人體姿態估計：從影像特徵到關鍵點與投籃動作量測	15
4.1	Pose Estimation 的任務定義	15
4.2	Pose Model 的完整推論流程	15
4.2.1	步驟一：人物定位與影像前處理	15
4.2.2	步驟二：Backbone 建立人體特徵表示	16
4.2.3	步驟三：Prediction Head 產生關節表示	16
4.3	Heatmap-based Pose：模型如何從機率圖找出關節？	17
4.3.1	Heatmap 解析度與量化誤差	17
4.4	座標回歸、Integral Regression 與 Coordinate Classification	18
4.5	多人姿態估計：Top-down 與 Bottom-up	19
4.6	幾種代表性 Pose Model 的設計重點	19
4.7	單一畫面中的可見性、遮擋與關節信心	20
4.7.1	關節信心如何進入後續量測？	20
4.8	影片中的 Pose：逐幀模型、Pose Tracking 與時間一致性	20
4.9	從關鍵點計算投籃量測	21
4.9.1	三點關節角度	21
4.9.2	球與手腕的關係，以及出手事件	21
4.10	Pose Model 的評估指標	22
5	專題 Proposal：從問題定義到驗證設計	23
5.1	從使用情境與問題定義開始	23
5.2	專題架構圖：呈現資料如何逐步轉換	24

5.2.1 每個模組至少需要留下哪些資訊？	24
5.3 錯誤傳遞：為什麼不能只量最後一個 KPI？	24
5.4 Baseline 與 Ablation：證明改進來自哪裡	25
5.4.1 Baseline 是合理的簡單方法	25
5.4.2 Ablation 是拆掉某個部件	25
5.5 資料切分：真正的泛化是跨影片、跨比賽、跨場景	25
5.6 成功案例與失敗案例要用同一套取樣規則	26
5.7 專題成果展示與口頭簡報	26
A 術語速查表	27
B 工具選擇矩陣	28
C 指標選擇速查	29
D 可延伸的專題方向	30
參考資料與進一步閱讀	30

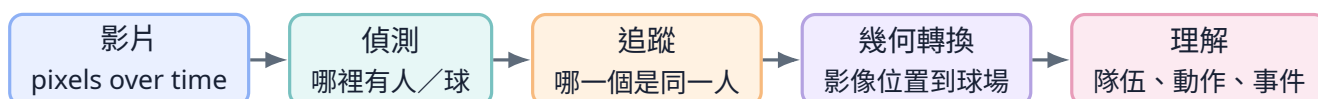
課程導論 | 籃球運動數據分析的核心問題

核心概念： 籃球電腦視覺不是單一模型，而是一條資料處理鏈。它把比賽影片轉成「誰、在什麼時間、位於球場哪裡、正在做什麼」的結構化資訊，再進一步回答站位、跑動、投籃與戰術問題。

1. 為什麼一段籃球影片，電腦看起來比人類困難很多？

人類看到影片時，會自動利用常識：知道橘色圓球是籃球、知道兩個穿同色球衣的人屬於同一隊、知道球員短暫被擋住後仍是同一個人，也知道球場線在透視畫面中其實代表固定的真實位置。

電腦沒有這些預設。對電腦而言，一張影像最初只是排列成矩陣的數字；影片則是連續很多張數字矩陣。要得到可用的籃球資訊，通常需要依序解決下列問題：



這條鏈上的每一步都可能出錯。 偵測少掉一位球員，追蹤就可能中斷；追蹤把兩個人交換身分，路徑統計就會錯；球場標定偏移，跑動距離就不可信；隊伍分錯，陣形與戰術分析也會跟著錯。因此，成熟的系統不只輸出答案，也必須保留中間結果與可信度。

2. 六項核心視覺任務

Classification | 影像分類

回答「這張圖主要是什麼」。例如判斷一張裁切圖是球員、裁判或籃球。輸出通常是類別，不告訴你物件在哪裡。

Object Detection | 物件偵測

回答「有哪些物件、各自在哪裡」。常用矩形框標示球員與籃球，是影片分析最常見的第一步。

Segmentation | 影像分割

回答「每個像素屬於什麼」。比矩形框精細，可分出球員輪廓、球場區域或球衣，但標註與運算成本通常較高。

Keypoint / Pose | 關鍵點與姿態

回答「重要的點在哪裡」。球場可標線段交點；人體可標肩、肘、腕、膝等關節，用於角度與動作分析。

Tracking | 追蹤

回答「這一幀的人，是否與上一幀是同一個」。追蹤的核心不是找到人，而是維持時間上的身分一致性。

Action / Event Understanding | 動作與事件理解

回答「正在發生什麼」。例如出手、傳球、掩護、換防或快攻。通常建立在偵測、追蹤、姿態與時序訊號之上。

3. 相關技術的發展脈絡

技術發展脈絡

第一階段：人工規則與傳統影像處理。 早期系統依賴顏色閾值、背景相減、邊緣、模板匹配與光流。固定相機、光線穩定時可以工作，但遇到陰影、遮擋、球衣相近或攝影機移動時容易失效。

第二階段：人工特徵 + 機器學習。 研究者設計 HOG、SIFT、Haar 等特徵，再搭配 SVM、AdaBoost 等分類器。這類方法比單純規則更能泛化，但特徵仍由人設計，對複雜運動場景的表現有限。

第三階段：深度學習偵測。 R-CNN 系列把深度特徵帶入物件偵測；SSD、YOLO 等單階段方法強調即時速度；之後多尺度特徵、anchor-free 設計與 Transformer detector 持續改善小物件與擁擠場景。

第四階段：Tracking-by-Detection。 先逐幀偵測，再用運動模型、框重疊與外觀特徵連接身分。SORT、DeepSORT、ByteTrack、BoT-SORT 都屬於這條路線，只是使用的關聯訊號與遮擋處理不同。

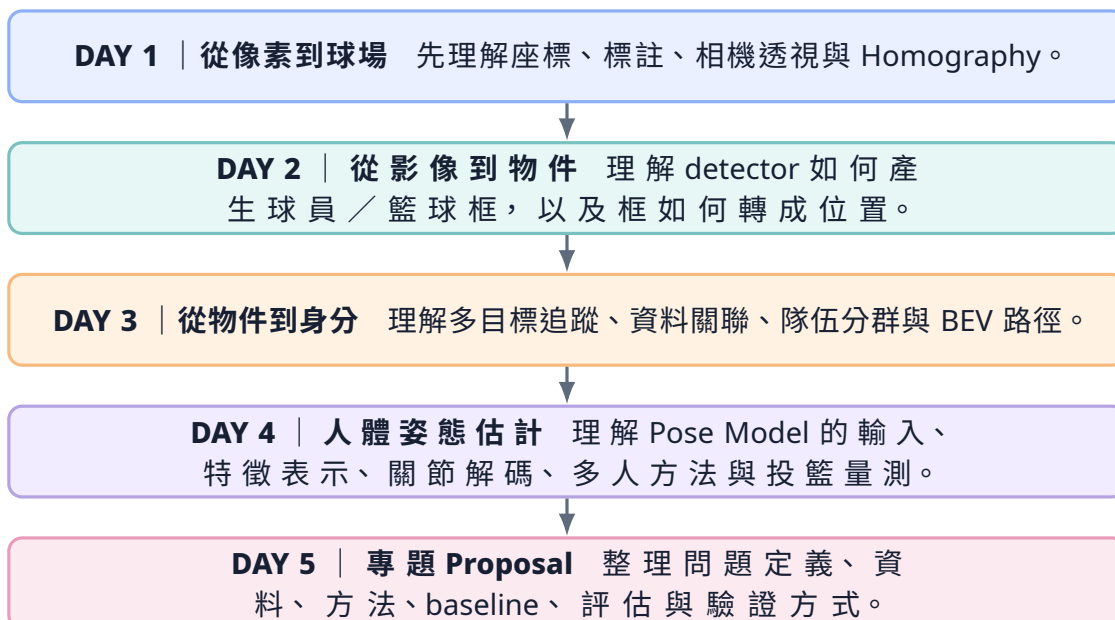
第五階段：端到端時序與多模態理解。 近年的方法開始使用 Transformer、長期記憶、跨鏡頭特徵、文字語意與大型視覺模型。但在實務上，模組化的 detection + tracking + geometry 仍最容易除錯、量測與部署。

4. 一張工具地圖：每個工具負責什麼？

常見工具與框架

工作階段	常見工具／框架	適合用途與提醒
資料標註	CVAT、Roboflow Annotate、Label Studio	建立 BBOX、polygon、mask、key-point。重點不是工具名稱，而是標註規則與版本管理。
幾何與影像處理	OpenCV、NumPy	Homography、相機標定、影像轉換、光流、濾波與視覺化。
模型訓練	PyTorch、Ultralytics、MMDetection、Detectron2	快速建立 detector；不同框架的資料格式、授權與部署方式不同。
多目標追蹤	ByteTrack、BoT-SORT、DeepSORT、OC-SORT	將逐幀偵測轉成 Track ID。應依相機是否移動、遮擋程度與即時需求選擇。
人體姿態	MediaPipe、MMPose、OpenPose、RTMPose	取得人體關鍵點。單人近距離與多人遠距離是不同難度。
評估與實驗	MOTChallenge tools、COCO metrics、Weights & Biases、MLflow	保存指標、失敗案例與實驗設定，避免僅依據少數展示案例判斷系統效能。

5. 本課程的五日路線圖



本節重點

這五天不是五個互不相干的模型。它們共同組成一條可以被檢查的系統：

video → detections → tracks → court positions → events → evidence

後面的分析品質永遠受前面模組限制，因此每一天都會同時談「能做什麼」與「什麼時候不能相信」。

從影像像素到真實球場：座標、標註與 Homography

核心問題： 攝影機記錄的是傾斜、有透視縮放的二維畫面，為什麼我們還能把球員的位置放回一張俯視球場圖？

完成這一天後，讀者應能： 分辨影像座標與球場座標、選擇適合的標註形式、理解 Homography 的假設與限制、知道如何檢查投影是否真的可靠。

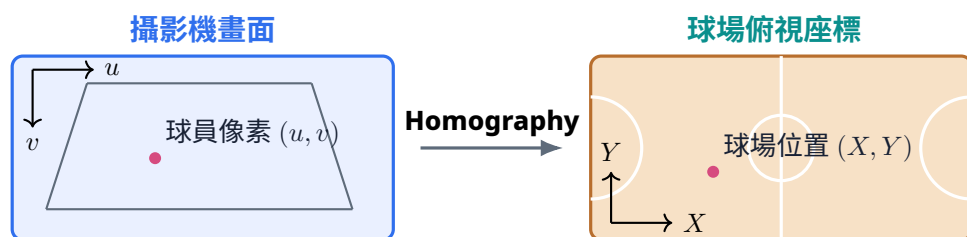
先建立直覺：相機看到的不是「距離」，而是「投影」

概念導入

人在球場上向遠處跑時，真實身高沒有變，但影像中的人會變小；平行的邊線在畫面中可能看起來逐漸靠近。這不是球場變形，而是三維世界經過相機投影後的結果。

影像座標通常以左上角為原點，水平向右為 u ，垂直向下為 v ，單位是像素。球場座標則可以用公尺表示，例如以球場左下角為 $(0,0)$ ，長邊為 X 、短邊為 Y 。兩者回答的問題不同：

- 影像座標回答：「這個框畫面中的哪一個位置？」
- 球場座標回答：「這位球員實際站在球場哪裡？」
- BEV (Bird's-Eye View, 鳥瞰圖) 是把場地轉成接近俯視的共同座標，方便比較距離、速度與站位。



資料標註：不是「框越精細越好」，而是要符合問題

四種常見標註及其代價

標註形式	表示方式	適合回答的問題	主要限制
影像分類	整張圖一個類別	這張 crop 是球、球員或裁判？	不知道物件位置。
BBOX	矩形框 (x_1, y_1, x_2, y_2)	哪裡有球員／球？	框中包含背景；無法表示精細輪廓。
Segmentation	每個像素的 mask	球衣區域、球員輪廓、球場區域	標註昂貴，模型與資料處理較複雜。
Keypoint	一組語意明確的點	球場交點、人體關節、籃框中心	點被遮擋時需要清楚的可見性規則。

籃球場標定最常用 **keypoint**。例如三分線與邊線交點、罰球線角點、中線與邊線交點。這些點在真實球場上有固定位置，因此可以建立「影像點 $\leftrightarrow$ 球場點」的對應關係。

標註規則比標註工具更重要

同一個「球員框」，不同標註者可能有不同理解：是否包含鞋子？跳躍時要不要框到影子？被遮擋一半時要框完整身體推測範圍，還是只框可見區？這些差異會形成 *label noise*，使模型學到互相矛盾的規則。

建議在正式標註前先建立 *annotation guideline*：

1. 定義每個類別，附上正例、反例與邊界案例。
2. 定義遮擋、截斷、模糊、小物件與畫面外物件的處理方式。
3. 先讓多人共同標一小批資料，檢查不一致處。
4. 每次修改規則都建立新的 dataset version，不覆蓋舊版。
5. Train / validation / test 依影片或比賽切分，避免相鄰幀洩漏。

常見誤解與失敗模式

最常見的資料洩漏： 同一支影片的連續影格非常相似。如果把第 100 幀放在訓練集、第 101 幀放在測試集，模型可能只是記住場景與球衣，而不是學會泛化到新比賽。

Homography：把一個平面映射到另一個平面

它能做什麼，也不能做什麼

Homography（單應性／平面投影轉換）描述兩個平面之間的射影關係。在籃球應用中，最常見的是把「攝影機中的球場平面」映射到「俯視球場平面」。

以齊次座標表示：

$$\lambda \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \quad H \in \mathbb{R}^{3 \times 3}.$$

H 有九個元素，但整體乘上一個常數不改變投影，因此只有八個有效自由度。每組二維點對提供兩條獨立方程，所以理論上至少需要四組不共線點對。

概念導入

把 Homography 想像成一張可彎曲但不能立體折疊的透明塑膠片。只要所有要映射的點都落在同一個平面上，就能把斜視球場拉回俯視圖；但球員頭部、飛在空中的球、籃框等不在地板平面上的點，無法被同一個 H 精確描述。

為什麼要用齊次座標？

普通二維矩陣乘法很難同時表示平移與透視除法。把點寫成 $[u, v, 1]^T$ 後，平移、旋轉、縮放、剪切與透視可以統一成矩陣乘法；最後再除以第三維得到一般座標：

$$\begin{bmatrix} \tilde{X} \\ \tilde{Y} \\ \tilde{W} \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \quad X = \frac{\tilde{X}}{\tilde{W}}, \quad Y = \frac{\tilde{Y}}{\tilde{W}}.$$

DLT、RANSAC 與重投影誤差

技術發展脈絡

DLT (Direct Linear Transform) 把點對關係改寫成線性齊次方程，再以 SVD 求解。它快速且常作為初始解，但對錯誤配對非常敏感。

RANSAC 反覆從資料中抽取最小樣本估計模型，再數有多少點符合模型。它的價值不是讓所有點都擬合，而是辨認哪些點可能是離群值。

重投影誤差可寫為：

$$e_i = \|\hat{\mathbf{q}}_i - \mathbf{q}_i\|_2, \quad \hat{\mathbf{q}}_i = \pi(H\mathbf{p}_i),$$

其中 $\pi(\cdot)$ 表示除以齊次尺度。除了平均誤差，也應觀察最大誤差、誤差分布與錯誤位置。

演算法流程（概念版）

```
INPUT: camera keypoints P, matching court points Q
REPEAT many trials
  randomly sample at least four non-collinear pairs
  estimate a candidate homography H
  project every camera point through H
  compute reprojection errors
  mark small-error pairs as inliers
KEEP the candidate with the strongest inlier support
REFIT H using all inliers
VALIDATE on independent court points, not only training pairs
```

標定點的選擇原則與品質檢查

- 點應分散在整個可見球場，不要全部集中在畫面中央。
- 避免幾乎共線；共線點無法穩定約束完整平面轉換。

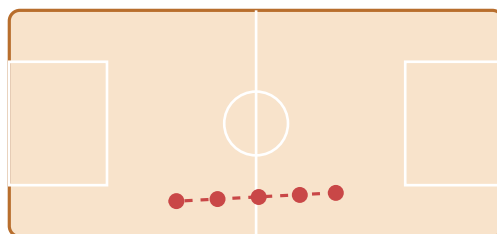
- 優先選輪廓清楚、定義唯一的線段交點。
- 不要把三分線弧上的模糊點當成精確交點，除非有明確幾何定義。
- 驗證點應與估計點分開，避免只檢查「用來算 H 的點」。

配置 A：覆蓋充分、幾何穩定



較穩定的原因：點分布在左右、遠近與內外區域，能同時約束尺度、旋轉、透視收斂與

配置 B：集中且近共線、容易不穩定



主要問題：雖然點數不少，但都落在短而近似直線的區域；球場上方與左右邊界缺少觀

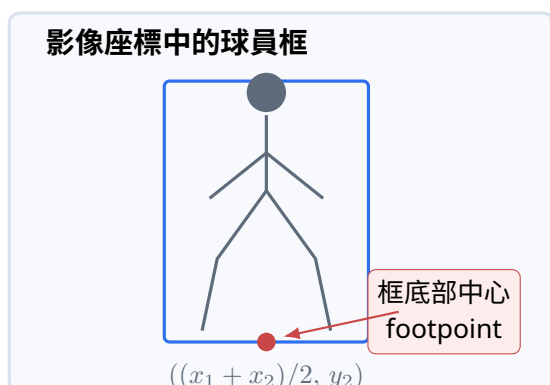
判斷重點不是「點越多越好」，而是覆蓋範圍、幾何方向、點位定義與獨立驗證是否完整。

從 BBOX 到球場位置：為什麼常取腳底中心？

球員的矩形框包含整個身體，但 Homography 假設點位於地板平面。最接近地面的點通常是鞋底，因此常用框的底部中心作為 footpoint：

$$\mathbf{f} = \left(\frac{x_1 + x_2}{2}, y_2 \right).$$

影像座標中的球員框



共同球場座標 (BEV)



這只是近似，以下情況會失效：

- 球員跳躍，鞋底離開地面。
- 框沒有包含腳，或被前景球員遮擋。
- 球員倒地、坐地或畫面只拍到上半身。
- 斜視角很強，框底部中心不等於兩腳接地位置。

更精細的方法可以使用人體腳踝 keypoint、鞋子 segmentation，或在多幀中平滑 footpoint；但模型更複雜不代表一定更準確，仍需以真實球場點誤差驗證。

工具選擇與實務流程

常見工具與框架

OpenCV： findHomography、perspectiveTransform、warpPerspective 是最常見的幾何工具。它們負責數值計算，不會自動判斷你選的點是否合理。

Roboflow / CVAT / Label Studio： 可建立 keypoint、BBOX、polygon 與資料版本。真正需要投入的是 annotation guideline、品質檢查與 split 策略。

NumPy / Pandas： 適合保存點對、計算誤差、輸出 CSV 與建立可重現的資料處理流程。

本日研究檢查表

本節重點

- 我是否清楚區分影像座標、BEV 像素與真實公尺？
- Homography 的點是否分散、非共線且定義一致？
- 是否有獨立驗證點，而不是只看估計用點？
- 是否把腳底接觸點與身體中心混為一談？
- 是否把平面方法誤用到飛行中的球或人體上半身？

課堂提問

為什麼「畫面中的框看起來很準」，投影到 BEV 後仍可能偏很多？請分別從 BBOX、footpoint、Homography 與透視放大效應解釋。

模型如何找到球員與籃球：Object Detection 與 BBOX-to-BEV

核心問題： 模型如何從數百萬個像素中，輸出「這裡有一名球員、那裡有一顆球」？它輸出的 confidence、BBOX 與類別究竟代表什麼？

先說需求：為什麼不能只做影像分類？

概念導入

分類模型可以告訴你「這張圖有籃球」，但無法指出籃球在哪裡。比賽畫面同時有多名球員、裁判與球，因此需要 Object Detection：一次輸出多個物件的位置與類別。

一個 detection 通常包含：

$$D_i = (x_1, y_1, x_2, y_2, c, s),$$

其中 (x_1, y_1, x_2, y_2) 是框、 c 是類別、 s 是模型分數。注意 s 通常不是經過統計校準的「真實正確率」，而是排序與篩選用的信心分數。

偵測技術的演進：從滑動視窗到端到端模型

技術發展脈絡

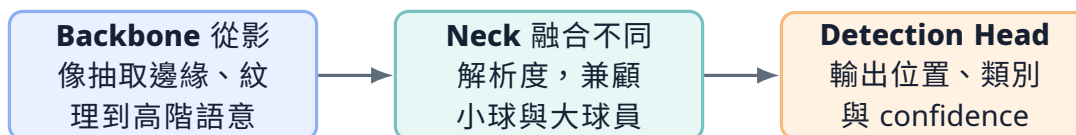
Sliding Window + 手工特徵： 在影像上移動許多不同大小的窗口，對每個窗口計算 HOG、Haar 等特徵，再用分類器判斷。問題是窗口數量巨大，而且特徵設計依賴人工經驗。

Two-stage detector： R-CNN、Fast R-CNN、Faster R-CNN 先找可能含物件的區域，再分類與微調框。通常準確，但流程較重。

One-stage detector： SSD、YOLO 將偵測直接視為密集預測，一次前向推論輸出大量候選框，適合即時影片。

Anchor-free 與 Transformer detector： 後續方法減少手工 anchor 設計，或使用 set prediction 與注意力機制。它們改變候選生成與匹配方式，但仍須面對小球、遮擋與資料偏差。

現代 Detector 的三個主要部件



低階特徵保留細節，高階特徵提供語意；
多尺度融合是小物件偵測的重要基礎。

為什麼籃球比球員難偵測？

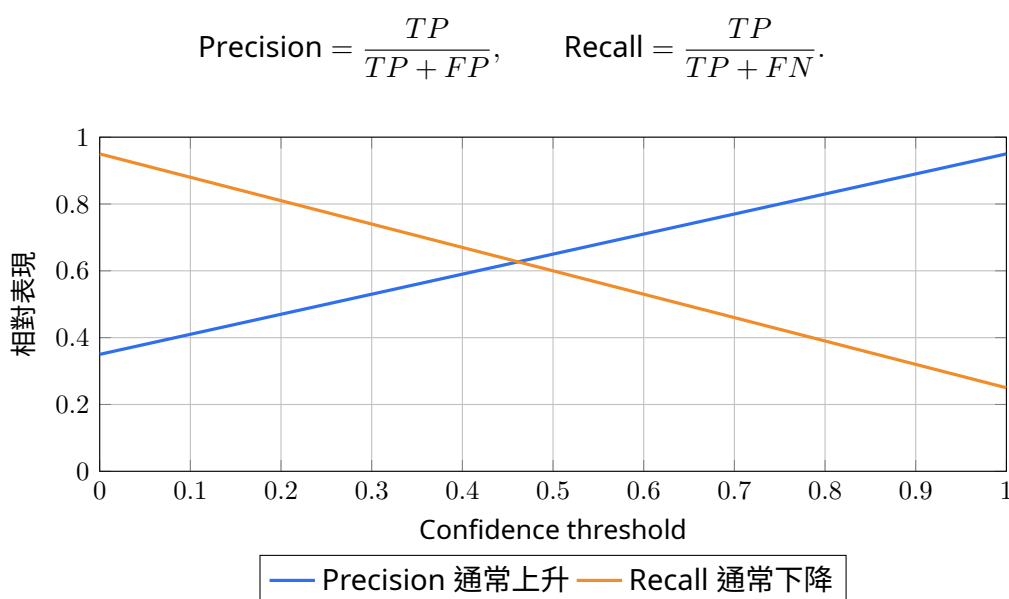
- 球在遠景中可能只有十幾個甚至更少像素。
- 高速運動造成 motion blur，球的邊界不再清楚。
- 球常被手、身體或籃框遮擋。
- 球的外觀會受光線、旋轉與壓縮影響。
- 訓練資料中「沒有球」的背景遠多於「有球」的位置，形成類別與尺度不平衡。

常見改善方向包括提高輸入解析度、增加遠距小球樣本、使用 crop / tiling、調整資料增強、保留高解析度特徵層、加入時序追蹤，或針對球建立專用 detector。

Confidence、Precision、Recall：三個常被誤解的概念

Confidence 門檻是一個取舍旋鈕

降低 confidence threshold 後，模型會保留更多候選：漏掉真實物件的機率下降，但誤報增加。提高門檻則相反。



這張圖是概念示意，不代表所有模型都呈線性關係。真正門檻應在 validation set 上，依應用成本決定。例如球追蹤常寧可先保留一些低分真球候選，再交給時序關聯排除背景。

IoU：框與框重疊多少

Intersection over Union：

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

IoU 可用於判斷預測框是否接近真值，也可用於 NMS 與 tracking association。但三種使用情境的 threshold 含義不同，不能混成一個參數。

NMS：為什麼同一個人會先出現很多候選框？

密集 detector 常在同一物件附近產生多個候選。Non-Maximum Suppression (NMS) 依分數排序，保留最高分框，再移除與它過度重疊的框。

演算法流程（概念版）

```

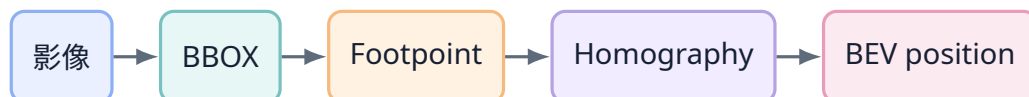
SORT all candidate boxes by confidence
WHILE candidates remain
  take the highest-score box as a kept detection
  compare it with the remaining boxes
  suppress boxes whose IoU is above the NMS threshold
RETURN all kept detections

```

常見誤解與失敗模式

在擁擠籃下，兩名球員的框可能高度重疊。NMS threshold 太低，可能把其中一人誤刪；太高，則同一人可能保留重複框。這是「去重」與「保留相鄰真實物件」之間的取捨。

從 Detection 到 BEV：資料鏈與誤差鏈



最終位置可以寫成：

$$\mathbf{q} = g(H, \mathbf{f}(B)).$$

因此誤差至少來自三處：

1. detector 框的位置誤差；
2. 從框推估 footpoint 的近似誤差；
3. Homography 本身與其空間外插誤差。

研究上應拆開評估。 只量最終 BEV 誤差，無法知道問題出在 detector 還是幾何；只量 mAP，也無法證明球場位置足夠準。建議同時保存 BBOX 指標、footpoint pixel error、Homography validation error 與最終公尺誤差。

常見工具與適用情境

常見工具與框架

- **Ultralytics YOLO**：適合快速訓練、推論與串接 tracking，API 簡潔；需留意版本、模型授權與預設參數。
- **MMDetection**：研究配置豐富，適合比較多種 detector 與 loss；學習曲線較高。
- **Detectron2**：適合 Faster R-CNN、Mask R-CNN 等研究與客製化。
- **TensorRT / ONNX Runtime**：用於部署與加速，不會自動提升模型本身的辨識能力。
- **Roboflow**：可協助資料管理、轉格式、訓練與部署；核心資料品質仍需自行負責。

Detection 結果的解讀與品質評估

本節重點

- 分類、定位與 confidence 是不同輸出。
- confidence 不是保證正確率，threshold 應依 validation 與應用成本設定。
- 球是小物件，資料尺度與時序資訊通常比單純換更大模型更重要。
- NMS 與 tracking association 是不同階段，IoU threshold 的含義不同。
- Per-frame detection 沒有身分，不等於 tracking。

從每一幀的框，到持續身分與隊伍：Multi-Object Tracking

核心問題： 當每一幀都有十幾個框時，系統如何知道上一幀的 7 號球員，到了下一幀仍是哪一個框？遮擋後重新出現，身分是否還能接回來？

什麼是追蹤？需求不是「畫軌跡線」而已

概念導入

Object Detection 每張圖都重新找物件；Tracking 則要維持時間上的一致身分。即使每一幀都偵測正確，只要框的順序不固定，系統仍不知道哪一段位置屬於同一位球員。

多目標追蹤（Multi-Object Tracking, MOT）的典型輸出是：

$$(t, \text{track_id}, x_1, y_1, x_2, y_2, \text{class}, \text{score}).$$

Track ID 是影片內的暫時識別碼，不等於球衣背號或真實姓名。要把 Track ID 對應到球員，通常還需要球衣 OCR、ReID、名單與人工校正。

追蹤技術的演進：從光流到資料關聯

技術發展脈絡

Template matching / Optical Flow： 追蹤前一幀的影像紋理如何移動。適合短時間局部追蹤，但外觀變化、遮擋與大位移時容易漂移。

Kalman Filter + Assignment： 以運動模型預測下一位置，再將 detections 與 tracks 配對。SORT 是代表性簡潔方法。

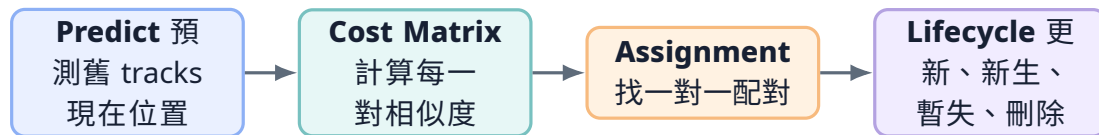
Appearance / ReID： DeepSORT 加入外觀 embedding，使系統不只看位置，也比較球員 crop 的視覺特徵，改善遮擋後的身分延續。

利用低分 detection： ByteTrack 不直接丟棄所有低分框，而用它們挽回被遮擋或模糊的既有軌跡。

相機運動與多線索融合： BoT-SORT 加入 camera motion compensation 與可選 ReID；OC-SORT 強化非線性與遮擋後的觀測修正。

Transformer / Memory-based MOT： 新方法嘗試用注意力與記憶直接建模長期關係，但成本、資料需求與部署複雜度通常更高。

Tracking-by-Detection 的完整流程



Motion model：先猜它會去哪裡

Kalman Filter 常以位置、尺度與速度建立狀態，根據前幾幀估計下一幀位置，再用 detection 修正。它不是「AI 認人」，而是帶有噪聲假設的動態估計器。

籃球的困難在於：急停、變向、跳躍、掩護與攝影機運動都會違反簡單等速度模型。因此，預測只能縮小搜尋範圍，不能單獨保證身分。

Cost matrix：每一格都是一個「可能是同一人嗎？」

假設上一幀有 m 條 tracks，這一幀有 n 個 detections，成本矩陣：

$$C \in \mathbb{R}^{m \times n}, \quad C_{ij} = 1 - \text{IoU}(T_i, D_j)$$

或融合多種線索：

$$C_{ij} = \alpha C_{ij}^{\text{motion}} + \beta C_{ij}^{\text{IoU}} + \gamma C_{ij}^{\text{appearance}}.$$

成本越低，越可能配對

	D1	D2	D3
T1	0.12	0.88	0.54
T2	0.91	0.08	0.47
T3	0.58	0.76	0.16

Hungarian assignment：不是每個 track 自己搶最近的框

Greedy 逐次選目前最小成本，速度快但可能讓早期選擇阻塞更好的整體配對。Hungarian algorithm 解線性指派問題，在一對一限制下找全域最小總成本。

即使使用 Hungarian，也需要 gating：如果距離太遠、IoU 太低或外觀差異太大，就應判定為不可能，而不是強迫配對。

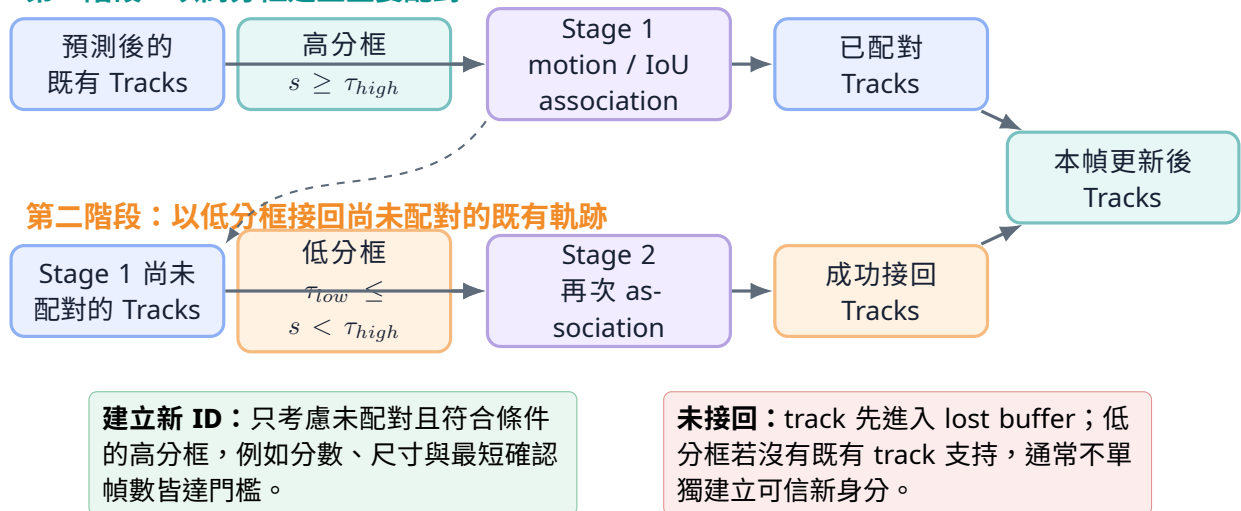
ByteTrack：為什麼低分框有時很有價值？

一般流程若在 tracking 前先移除所有低 confidence detections，被遮擋、模糊或只露出部分身體的真實球員可能被一起丟掉，造成 track 斷裂。

ByteTrack 的核心是兩階段關聯：

- 步驟 1：** 依 confidence 將 detections 分成高分組與低分組。
- 步驟 2：** 先預測既有 tracks 在本幀可能出現的位置。
- 步驟 3：** 第一階段只用高分框建立主要配對，取得穩定更新與尚未配對的 tracks。
- 步驟 4：** 第二階段讓尚未配對的 tracks 再與低分框比對，嘗試接回遮擋或模糊造成的弱觀測。
- 步驟 5：** 只有符合條件的未配對高分框可以建立新 track；低分框通常不能單獨建立新身分。
- 步驟 6：** 暫時未匹配的 track 進入 lost buffer，超過保留時間後才移除。

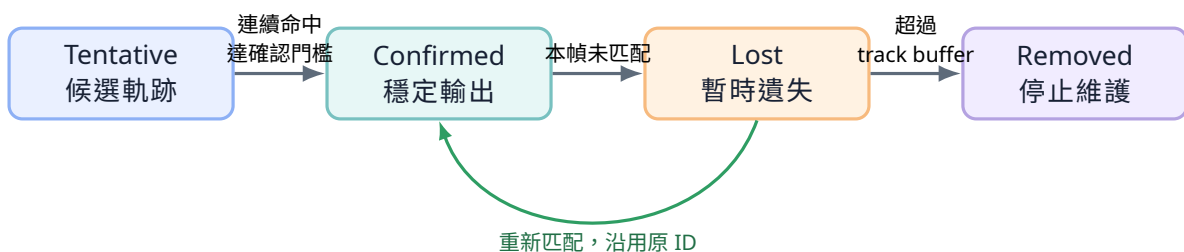
第一階段：以高分框建立主要配對



場上例子： 一名進攻球員從掩護者身後通過，連續數幀只露出半個身體。Detector 的 confidence 可能從 0.86 降到 0.28，但框的位置仍與原 track 的預測區域重疊。第二階段可利用這個低分框維持原 ID。相反地，畫面角落突然出現一個 0.28 的低分框時，系統不應立刻把它當成新球員。

Track lifecycle：身分是有生命週期的

track 不會因為單幀出現就立刻成為可靠身分，也不會因為漏掉一幀就立即消失。生命週期用來累積證據、容忍短期遮擋，並限制錯誤軌跡無限延續。



常見狀態：

狀態	意義	為什麼需要
Tentative	剛建立、尚未累積足夠證據	避免單幀誤報立刻成為穩定 track。
Active / Confirmed	持續被 detection 支持	可輸出並進入分析。
Lost	暫時未匹配，但仍保留	容忍短期遮擋或漏偵。
Removed	超過 buffer，正式刪除	避免無限保留過期狀態。

常見誤解與失敗模式

Track buffer 太短：遮擋一下就換 ID。**Track buffer 太長**：舊 track 可能錯接到另一名球員。參數必須配合 FPS、遮擋持續時間與球員密度，而不是照抄預設值。

ReID 與隊伍辨識：位置不夠時，加入外觀

Re-identification 是什麼？

ReID 模型把球員裁切圖轉成 embedding，使同一人的特徵距離較近、不同人的較遠。它可以改善長遮擋與再次出現，但籃球場景有特殊困難：同隊球衣幾乎相同、背號尺寸小、姿態變化大、正反面外觀差異明顯。

因此 ReID 不應被理解為「臉部辨識」。它多半只是輔助 association 的外觀線索。

無標籤隊伍分群

最簡單做法是裁切 torso，抽取 HSV histogram，再用 K-means 分成兩群：

$$\min_{\{c_i\}, \{\mu_k\}} \sum_i \|\mathbf{x}_i - \mu_{c_i}\|_2^2.$$



單幀分群容易受遮擋、陰影與背景影響。更可靠的方法是對同一 track 收集多幀特徵，再平均或投票；如果有已知隊伍標籤，supervised classifier 通常比 K-means 更可控。

常見誤解與失敗模式

Cluster 0 / Cluster 1 沒有固定語意。K-means 重新執行後編號可以交換；必須用參考顏色、已知球員或人工規則將群組對應到真正隊名。

Tracking-to-BEV：把時間與空間合併

每筆資料可保存：

$$(t, \text{track_id}, B_t, \mathbf{f}_t, \mathbf{q}_t, \text{team}, \text{confidence}).$$

有了持續身分與球場位置後，才能計算：

- 路徑長度與速度；
- 球員間距與 spacing；
- 進攻／防守佔位；
- 禁區停留、底角使用率；
- 換防、協防、掩護與 transition 的候選事件。

路徑長度：

$$L = \sum_{t=2}^T \|\mathbf{q}_t - \mathbf{q}_{t-1}\|_2.$$

但實作前需處理 FPS、單位、漏幀、插值、ID switch 與不合理跳點。若 ID 在中途交換，視覺上平滑的軌跡仍可能包含嚴重錯誤。

追蹤應該怎麼評估？

指標	看的是什麼	不足之處
MOTA	綜合漏偵、誤報與 ID switch	受 detection 表現強烈影響。
IDF1	身分配對的一致性	對軌跡局部結構的描述有限。
HOTA	同時考慮 detection 與 association	較完整，但需理解分解結果。
ID switches	身分交換次數	數量少不代表每次影響都小。
Track coverage	每位真實球員有多少時間被穩定覆蓋	適合實務報告，但需自行明確定義。

本節重點

追蹤的核心是 **data association**。Detector 決定有哪些候選，motion model 提供位置先驗，appearance 提供外觀線索，assignment 建立一對一關係，lifecycle 管理暫時遺失。任何一部分過度自信，都可能造成 ID switch。

人體姿態估計：從影像特徵到關鍵點與投籃動作量測

本日學習主軸： 理解 Pose Model 如何把 RGB 影像轉成肩、肘、腕、髁、膝、踝等關鍵點；比較 heatmap、座標回歸、top-down 與 bottom-up 等主要方法；最後再將關鍵點轉成投籃角度、時序與出手事件。

Pose Estimation 的任務定義

人體姿態估計 (Human Pose Estimation) 不是直接辨識「這個人正在投籃」，而是先預測一組具有固定語意的身體關鍵點。例如 COCO 人體格式常用鼻子、肩、肘、腕、髁、膝與踝等 17 點；MediaPipe Pose 則使用較細的 33 個 landmarks。不同資料集與模型的關節定義並不完全相同，因此在比較模型或串接程式前，第一步是確認 keypoint index、左右方向與座標定義。

對第 k 個關節，二維模型的輸出可寫成：

$$\hat{\mathbf{p}}_k = (\hat{x}_k, \hat{y}_k), \quad c_k \in [0, 1],$$

其中 c_k 表示可見度或模型信心。三維模型可能再輸出深度 $\hat{z}_k$ ，但 z 的原點、尺度與座標系必須另行確認。



骨架線不是模型直接看到的骨頭。 模型通常只輸出一組離散關鍵點；肩到肘、肘到腕等線段是依資料集預先定義的 skeleton connections 畫出來，方便閱讀與後續計算。

Pose Model 的完整推論流程

步驟一：人物定位與影像前處理

Top-down pose 系統會先使用 person detector 找到人物框，再把每個框裁切成固定比例的輸入影像。這一步看似只是 resize，實際上會直接影響關節位置：若人物框切掉手腕或腳踝，後面的 pose model 沒有足夠影像證據；若任意拉伸長寬比，人體比例也會改變。

常見前處理包括：

- 擴張人物框，使手腳不容易落在 crop 邊界；
- 依模型輸入比例做 letterbox 或 affine transform；
- 將 pixel 轉為 tensor，進行 normalization；
- 保存 affine matrix，將預測點由模型輸入座標映回原始影像。

步驟二：Backbone 建立人體特徵表示

Backbone 的工作不是直接輸出關節，而是把影像轉成 feature maps。淺層特徵保留邊緣、顏色與局部紋理；深層特徵逐漸整合手臂、軀幹與全身的語意關係。Pose estimation 對空間位置非常敏感，因此模型不能只保留「影像裡有人」的語意，也要保留「手腕究竟在哪一個像素附近」的高解析度資訊。

HRNet 的代表性設計是在網路中持續保留高解析度分支，並反覆與低解析度語意特徵交換資訊；ViTPose 則使用 Vision Transformer 作為 encoder，再以較輕的 decoder 產生關鍵點表示。兩者都在處理同一個核心矛盾：**辨識關節需要大範圍上下文，精確定位又需要高空間解析度。**

步驟三：Prediction Head 產生關節表示

Pose head 最常見的輸出形式包括：

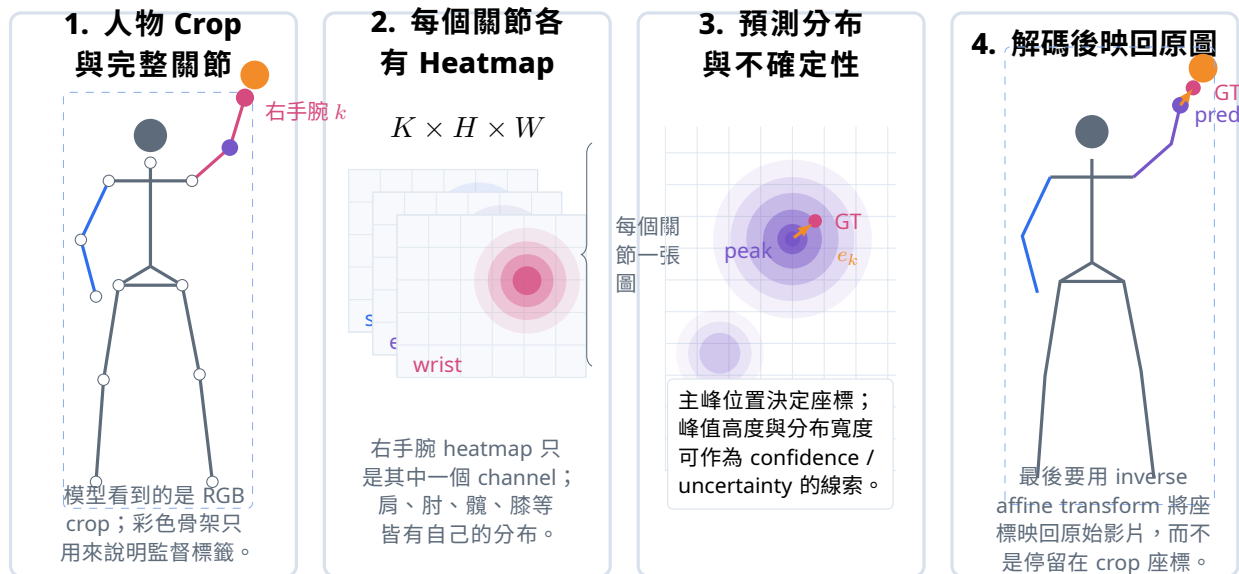
1. 每個關節一張二維 heatmap；
2. 直接回歸 (x, y) 座標；
3. 將 x 與 y 分別轉成一維座標分類；
4. 同時輸出關鍵點、可見度、深度或關節間連接資訊。

Heatmap-based Pose：模型如何從機率圖找出關節？

Heatmap-based pose 不只輸出一個點，而是對每一個關節建立一張二維分布。若資料集定義 K 個關節，模型輸出通常可視為 $K \times H \times W$ 的張量：第 k 張 heatmap 專門描述第 k 個關節在輸入 crop 中可能出現的位置。以右手腕為例，訓練標籤會在真實位置附近建立 Gaussian peak：

$$H_k(x, y) = \exp\left(-\frac{(x - x_k)^2 + (y - y_k)^2}{2\sigma^2}\right).$$

模型學習輸出 $\hat{H}_k$ ，再以 MSE、KL divergence 或其他 heatmap loss 使預測分布接近標籤。推論時可用 argmax、soft-argmax 或分布期望值取得座標。



讀圖重點： 每個關節對應獨立的 heatmap channel；分布的 peak、寬度與次峰反映座標和不確定性；解碼後的座標還必須經 inverse affine transform 映回原始影像，才能與人物 Track、球的位置及完整骨架共同分析。

Heatmap 解析度與量化誤差

若輸入影像是 256×192 ，輸出 heatmap 只有 64×48 ，一個 heatmap pixel 對應原圖約 4×4 pixels。直接 argmax 會把座標限制在離散格點上，因此常見 quarter-pixel correction、soft-argmax、DARK decoding 或 integral regression 來降低量化誤差。

Heatmap 解析度提高通常能改善定位，但也會增加顯存與計算量。對遠距籃球員而言，手腕在原圖中可能只有數個 pixels；若先經 detector crop，再縮到低解析度輸入，資訊可能已經不足，不能只靠 decoder 補回。

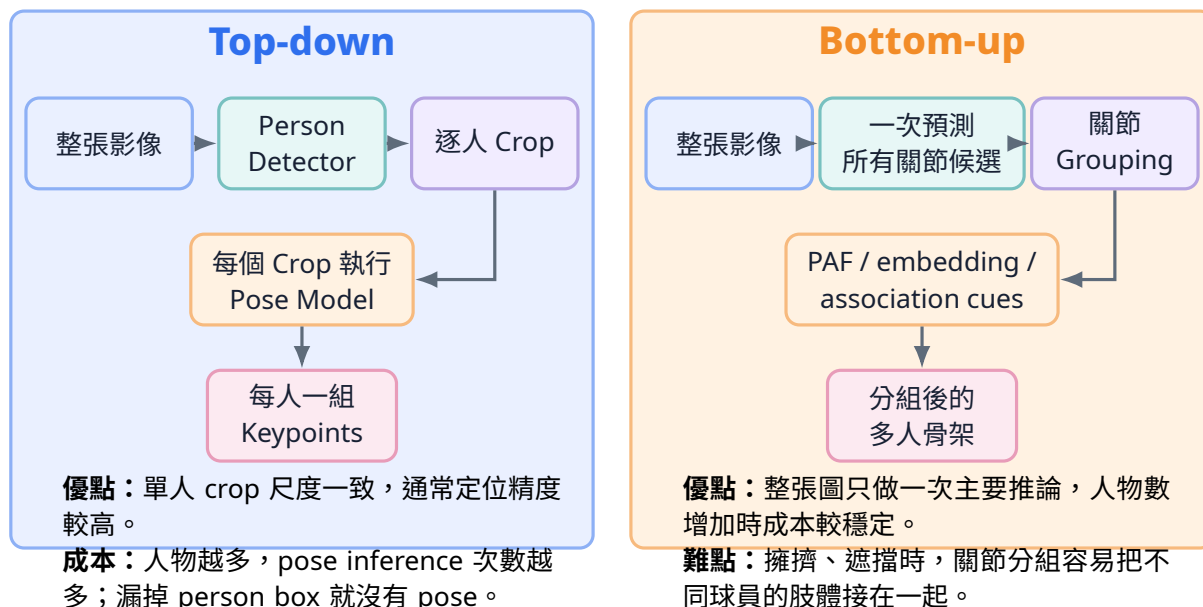
座標回歸、Integral Regression 與 Coordinate Classification

表示方式	核心做法	特性
2D Heatmap	每個關節預測一張 $H \times W$ 機率圖	直觀、定位效果佳；高解析度輸出成本較高，存在量化誤差。
Direct Regression	網路直接輸出 $2K$ 個座標	輸出緊湊，但從整體 feature 直接回歸精細位置較困難。
Integral Regression	對 heatmap 做可微分期望值，取得連續座標	結合 heatmap 表示與可微分座標回歸，可減少 argmax 不可微問題。
Coordinate Classification	將 x 、 y 分別切成 bins，預測兩個一維分布	避免完整二維 heatmap；SimCC 與 RTMPose 採用此類思路，兼顧精度與效率。

選擇輸出表示時要同時考慮： 模型輸入解析度、人物在畫面中的大小、部署速度、關節定位精度，以及後續是否需要手腕速度、角度差分等對抖動敏感的量測。

多人姿態估計：Top-down 與 Bottom-up

多人畫面除了要找出所有關節，還要回答「這個手腕屬於哪一位球員」。目前常見方法可分為兩條主路線。



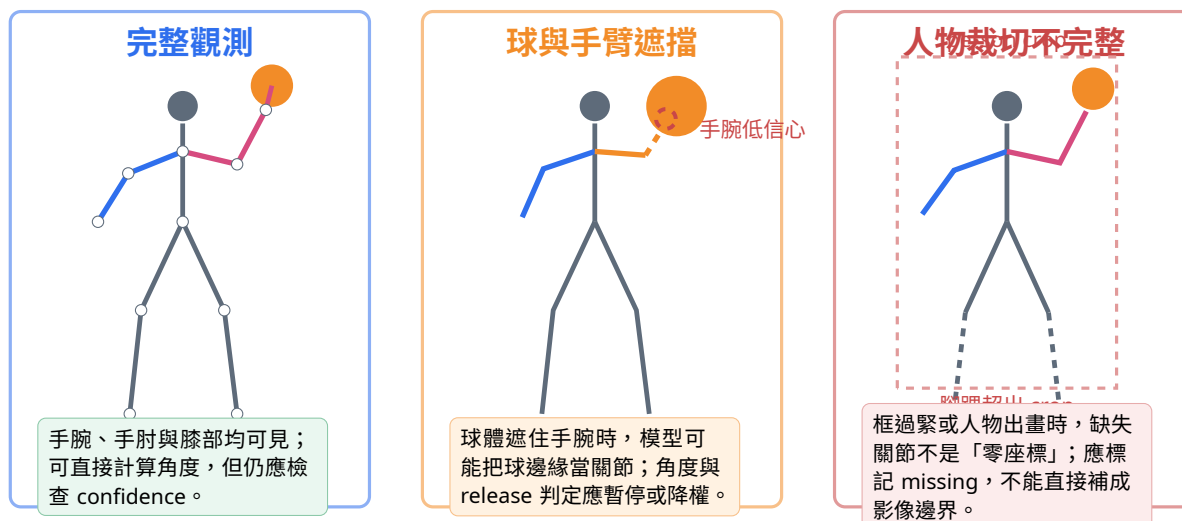
在籃球比賽中，球員數量固定但遮擋頻繁。Top-down 方法較容易與既有 detector、tracker 串接：先利用 track 的人物框裁切，再估計該 track 的 pose。若是全場遠距畫面，人物 crop 的有效解析度仍可能過低；此時即使 detector 找得到球員，手腕與腳踝也未必有足夠像素可辨認。

幾種代表性 Pose Model 的設計重點

模型／家族	主要路線	重要設計	在籃球情境的解讀
OpenPose	Bottom-up	同時預測 keypoint heatmaps 與 Part Affinity Fields，再把關節組成人。	可直接處理多人，但擁擠交錯與小人物時 grouping 較具挑戰。
HRNet	常作 Top-down backbone	維持高解析度分支並反覆做 multi-scale fusion。	適合需要精確定位的關節；計算量取決於模型規模與輸入尺寸。
BlazePose	單人 detector + tracker	輕量化、行動裝置即時、33 landmarks。	適合單人訓練與近距離鏡頭，不應直接假設可處理全場多人比賽。
ViTPose	Top-down Transformer	/ 以 plain Vision Transformer 建立可擴充的人體表徵，再由輕量 decoder 預測 key-points。	精度與模型規模彈性大；部署成本需依 backbone 尺寸評估。
RTMPose	即時 Top-down	強調訓練、架構與部署整合，常搭配 coordinate classification。	適合需要較高吞吐量的多人 pipeline，但仍受 person crop 解析度限制。

單一畫面中的可見性、遮擋與關節信心

Pose model 的輸出不是一副永遠完整的骨架，而是每個關節的座標與可信程度。即使人物框偵測正確，手腕、手肘、膝蓋或腳踝仍可能因為球體遮擋、肢體互相重疊、人物出畫或裁切範圍太緊而失去可靠觀測。因此，後續的投籃角度與事件分析不能只讀取座標，還必須同時使用 keypoint confidence 與可見性規則。



關節信心如何進入後續量測？

對關節 k ，模型通常輸出 (x_k, y_k, s_k) ，其中 s_k 是 confidence。角度或事件規則應先檢查參與計算的每個關節：

1. 若肩、肘、腕任一點低於門檻，該幀的手肘角度標記為缺值，而不是強行計算。
2. 若只有一兩幀短暫遺失，可使用前後幀與速度限制進行插值；長時間遮擋則保留缺值。
3. 若關節位於 crop 邊界附近，應額外降低可信度，因為截斷常造成系統性偏差。
4. 做時間平滑時，同時輸入 confidence，讓高信心觀測影響較大、低信心觀測影響較小。

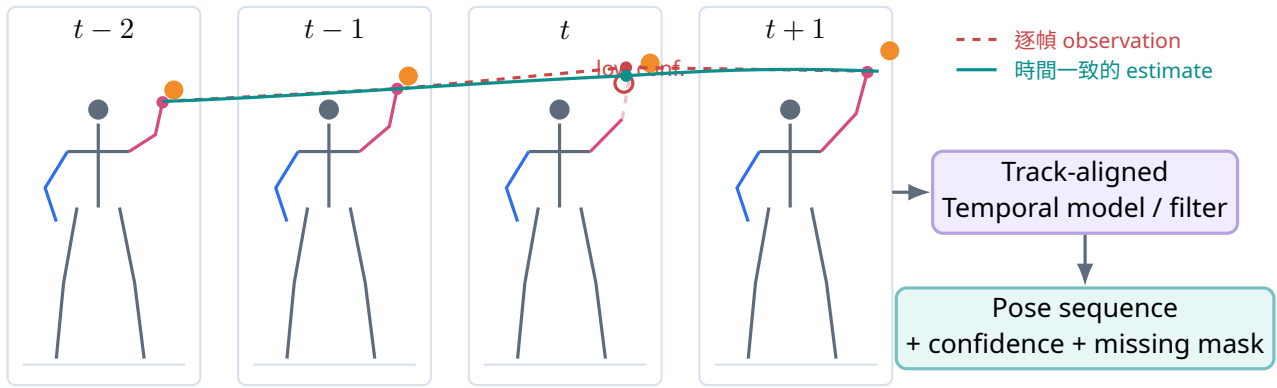
資料表示建議： 每個關節至少保存 x 、 y 、confidence、visible/missing 與原始影像尺寸。缺失關節使用明確的 mask 欄位，不要用 $(0, 0)$ 代替，否則後續平均與角度計算會產生不存在的動作。

影片中的 Pose：逐幀模型、Pose Tracking 與時間一致性

單張 pose model 通常獨立處理每一幀，因此關節會因影像模糊、遮擋與 detector crop 波動而跳動。影片系統常加入：

- 以人物 Track ID 固定同一人的 crop；
- 使用上一幀關節位置縮小搜尋範圍；
- 以 EMA、Savitzky-Golay 或 Kalman filter 降低高頻抖動；
- 使用 temporal convolution、RNN 或 video Transformer 直接學習時間關係；

- 對低 confidence 關節保留缺值，不任意連線或補值。



紅色虛線是逐幀手腕觀測；青色曲線是利用前後幀、Track ID 與 confidence 得到的時間估計。低信心關節應保留 missing mask，而不是強行連成一條看似平滑但缺乏證據的曲線。

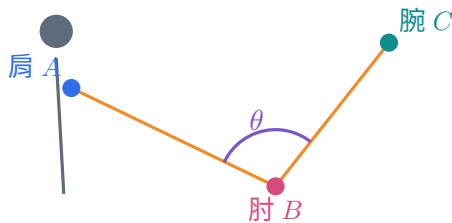
時間模型的目標不是把動作全部磨平。 投籃出手本身包含快速加速與方向改變；過強的 smoothing 會推遲 release frame、降低峰值速度。評估時應同時量測 jitter、事件時間誤差與動作峰值保留程度。

從關鍵點計算投籃量測

三點關節角度

給定肩 A 、肘 B 、腕 C ，以 B 為關節中心：

$$\theta = \cos^{-1} \left(\frac{(A - B) \cdot (C - B)}{\|A - B\|_2 \|C - B\|_2} \right).$$

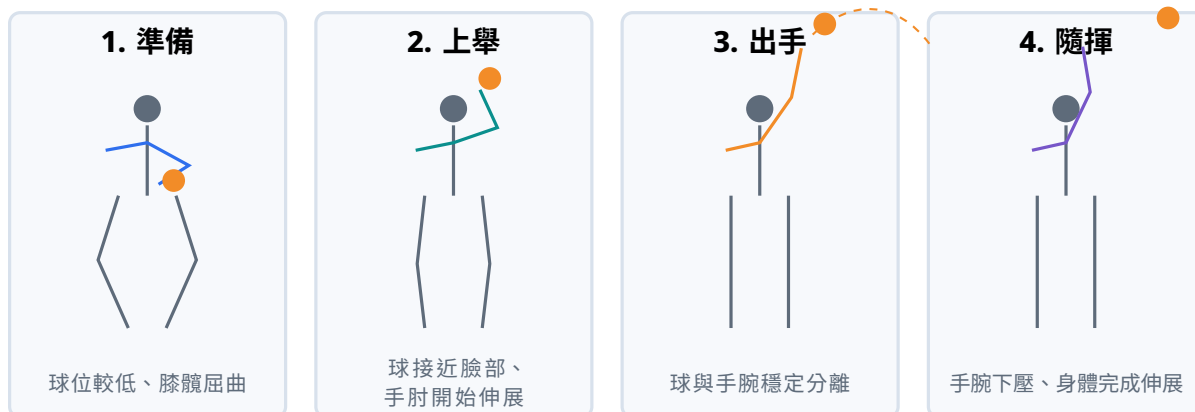


角度穩定性取決於三個點。若手腕只有 3-5 pixels 的跳動，經向量與反餘弦運算後，角度曲線可能產生更大的尖峰。計算前應先檢查 confidence、骨段長度與時間連續性。

球與手腕的關係，以及出手事件

出手 (release) 不是由 pose model 直接輸出的標準關節。它通常需要結合 pose、ball detection / tracking 與時間規則：

1. 找出持球者 track，取得投籃手腕位置；
2. 追蹤球中心，計算 ball-to-wrist distance；
3. 檢查距離是否持續增加，而不是單幀誤差；
4. 同時觀察球速度方向、手肘伸展與手腕隨揮；
5. 將候選區間與人工標註比較，輸出時間誤差與信心。



演算法流程（概念版）

```

FOR each tracked player and each frame
  transform the player crop and run the pose model
  map keypoints back to the original image
  reject or mask low-confidence joints
  associate the ball track with the shooting hand
  compute joint angles, ball-to-wrist distance and temporal derivatives
  FIND a stable release interval using several consecutive frames
  RETURN the interval, supporting signals and confidence
  
```

Pose Model 的評估指標

指標	量測內容	使用時需確認
PCK	預測點距離真值小於指定比例門檻的比率	距離用 bounding box、頭部尺寸或 torso 尺度正規化？門檻是多少？
OKS	依人物尺度與不同關節容許誤差，計算 keypoint similarity	COCO keypoint AP 以 OKS 為基礎；不同關節的 κ 不同。
Keypoint AP / AR	在多個 OKS threshold 下評估 precision 與 recall	同時受到人物偵測、關節定位與多人分配影響。
MPJPE	三維關節預測與真值的平均歐氏距離	是否 root-aligned、scale-aligned 或 Procrustes aligned，數值不可直接混比。
Temporal error	出手幀、事件起訖或峰值時間的誤差	應以 frame 或 millisecond 表示，並記錄影片 FPS 與標註一致性。

本節重點

Pose Model 的核心流程是：人物區域與座標轉換 → 特徵抽取 → 關節表示 → 解碼座標 → 時序與幾何量測。模型架構只決定其中一部分；人物尺度、視角、crop、heatmap 解析度、多人關節分配與時間一致性，會共同決定籃球動作分析是否可靠。

專題 Proposal：從問題定義到驗證設計

核心問題： 如何把前四天的 detector、tracker、geometry、pose 與事件模組，整理成問題明確、資料可追溯、方法可比較、結果可驗證的專題 Proposal？

從使用情境與問題定義開始

概念導入

「使用 YOLO、ByteTrack 與 MediaPipe 建立籃球分析系統」仍只是技術組合，尚未形成完整的專題問題。專題 Proposal 應先說明：研究或使用對象是誰、資料來自什麼情境、系統要輸出什麼，以及成果將如何被驗證。

一個可評估的研究問題通常包含：

- **對象：**球員、教練、分析師、轉播團隊或研究者。
- **情境：**固定相機、轉播畫面、單人訓練、固定半場比賽影片。
- **輸入：**RGB 影片、球場 keypoints、名單與必要的標定資料。
- **輸出：**位置、身分、隊伍、事件、動作量測或報表。
- **比較：**比哪個 baseline 更好？改善什麼指標？
- **限制：**什麼視角、光線與遮擋條件下不保證？

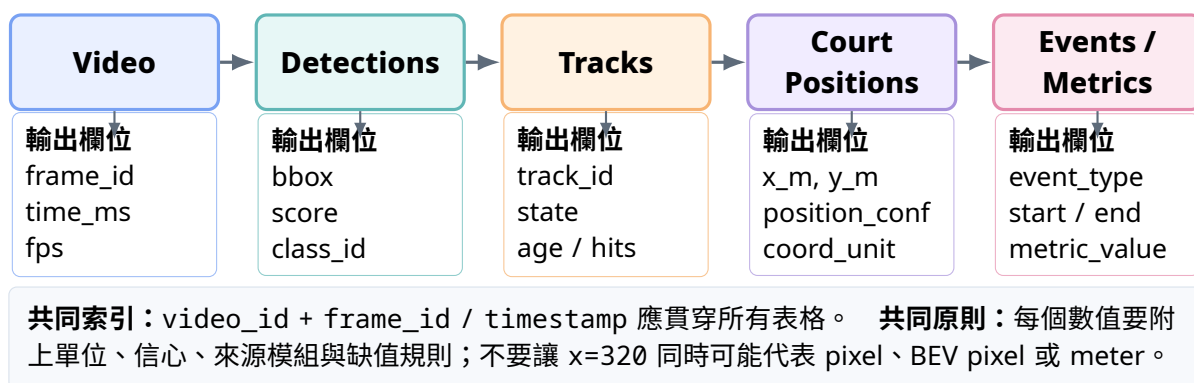
範圍過廣：用 AI 做球員分析。

可驗證的研究問題：在固定半場相機下，track-level 球衣特徵投票是否比單幀 K-means 降低隊伍標籤切換率？

具體實驗版本：在三場不同球衣與光線的測試影片上，比較單幀 HSV、單幀深度 embedding 與 track-level temporal voting 的隊伍分類準確率與切換率。

專題架構圖：呈現資料如何逐步轉換

專題架構圖應讓讀者看見資料在每個模組中如何改變，而不只是列出所使用的模型名稱。模組名稱、輸入輸出欄位與座標單位應該能互相對應；當最終結果不合理時，教師與學生才能沿著資料流追查是哪一步出錯。

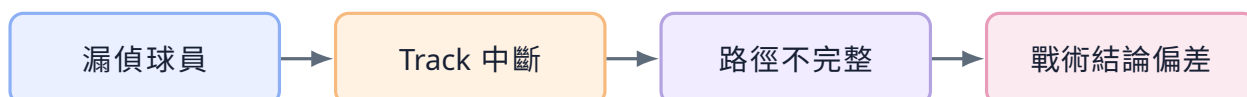


每個模組至少需要留下哪些資訊？

模組	建議保存內容	發生異常時先檢查
Video	FPS、解析度、時間戳、相機 ID、是否重新編碼	掉幀、可變 FPS、時間戳不一致。
Detection	BBOX、class、score、模型版本與影像尺寸	閾值、縮放方式、NMS 與漏偵位置。
Tracking	track ID、狀態、hits、age、match cost	ID switch、buffer、gating 與 detector 波動。
Court position	footpoint、Homography 版本、X/Y 單位、誤差	點位定義、相機移動、外插區域。
Event / metric	事件時間窗、使用訊號、品質旗標、最終數值	前置模組缺值、平滑延遲、規則邊界。

具體資料列範例：在第 1,248 幀，固定半場影片中的 track_id=7 由 detection 取得 BBOX，footpoint 經 homography_v3 映射為 (12.4, 5.8) meter，位置可信度為 0.81。這樣的紀錄比只保存一張畫好軌跡的影片更容易重現、比較與除錯。

錯誤傳遞：為什麼不能只量最後一個 KPI？



因此應設計 intermediate metrics：

模組	建議指標	對最終系統的影響
Detection	Precision、Recall、mAP、小物件 recall	漏人、漏球與誤報。
Tracking	IDF1、HOTA、ID switch、coverage	路徑是否屬於同一人。
Geometry	validation reprojection error、court position error	距離、速度與陣形是否可信。
Team / Identity	accuracy、confusion matrix、switch rate、reject rate	隊伍與球員統計是否被污染。
Pose / Event	PCK、角度誤差、release frame error	動作與事件時間是否準確。
End-to-end	任務完成率、分析誤差、使用者判讀時間	系統是否真的有用。

Baseline 與 Ablation：證明改進來自哪裡

Baseline 是合理的簡單方法

例如研究 track-level team voting：

- Baseline A：單幀 HSV + K-means。
- Baseline B：單幀 deep embedding + classifier。
- Proposed：多幀 embedding 平均 + confidence-weighted voting。

Ablation 是拆掉某個部件

- 去掉 temporal voting，看改善是否真的來自時間整合。
- 去掉 torso crop，看背景抑制是否重要。
- 去掉 confidence weighting，看簡單多數決是否已足夠。
- 將 ByteTrack 換成 SORT，看 low-score association 的貢獻。

常見誤解與失敗模式

只比較「我們的方法 vs 一個很差的設定」無法證明有效。Baseline 應該是合理、可重現且公平調參的替代方法；所有方法要使用相同 test split 與評估流程。

資料切分：真正的泛化是跨影片、跨比賽、跨場景

- 不要把同一段連續影片隨機拆成 train/test。
- 若目標是跨比賽泛化，test 應來自未見比賽。
- 若目標是跨場館泛化，test 應包含未見背景、鏡頭與光線。
- 若目標是跨球衣，test 應包含未見顏色與樣式。

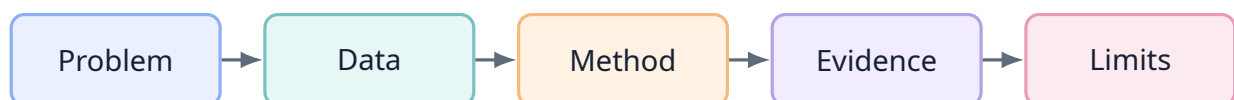
- 所有 threshold 與參數只能在 validation set 調整，不能看 test 再修。

成功案例與失敗案例要用同一套取樣規則

建議建立 failure taxonomy：

Detection failure
漏掉遠處球、把觀眾當球員、球與頭部混淆。
Association failure
遮擋後換 ID、兩人交叉時身分交換、舊 track 錯接新球員。
Geometry failure
標定點錯、相機移動後仍使用舊 H 、foot-point 被遮擋。
Identity / Team failure
相同球衣、裁判混入、陰影造成顏色偏移、背號不可見。
Pose / Event failure
手腕被球遮住、多人 pose 對錯人、release 規則對某種投籃失效。
Data / System failure
FPS 讀錯、時間戳不同步、單位混用、資料欄位版本不一致。

專題成果展示與口頭簡報



1. 先展示一個具體使用情境，而不是先講模型架構。
2. 明確定義輸入、輸出與成功條件。
3. 架構圖說明資料如何流動。
4. 用表格與曲線回答「是否比 baseline 好」。
5. Demo 顯示代表性情況，不取代正式評估。
6. 主動展示失敗案例與下一個最值得做的實驗。

術語速查表

英文術語	中文角色	直觀解釋
Pixel	像素	影像中最小的顏色格子；模型最初看到的是像素數值。
Frame	影格	影片中的一張靜態影像。
FPS	每秒影格數	決定時間解析度；30 FPS 表示每秒 30 張。
BBOX	邊界框	用矩形近似物件位置。
Keypoint	關鍵點	有明確語意的點，例如手腕、球場線交點。
Segmentation mask	分割遮罩	指出物件每個像素的精細區域。
Confidence	信心分數	模型內部的排序分數，不一定是校準後正確率。
IoU	交並比	兩個框重疊面積除以聯集面積。
NMS	非極大值抑制	移除同一物件附近的重複候選框。
Homography	平面單應性	將一個平面的座標映射到另一個平面。
BEV	鳥瞰圖	將場地轉為接近俯視的共同座標。
Footpoint	腳底接觸點	用來代表球員落在地板上的位置。
Tracking	追蹤	跨影格維持同一物件的身分。
Track ID	軌跡識別碼	影片內暫時身分，不等於球員姓名或背號。
Data association	資料關聯	決定哪個 detection 應接到哪個 track。
Kalman Filter	卡爾曼濾波	用動態模型預測狀態，再用觀測修正。
Hungarian algorithm	匈牙利指派	在一對一限制下尋找最小總成本配對。
ReID	再識別	用外觀 embedding 協助遮擋後重新連接身分。
Embedding	特徵向量	將影像 crop 轉成可比較距離的向量。
Clustering	分群	沒有標籤時，依特徵相似度自動分組。
Pose estimation	姿態估計	預測人體關節位置。
Reprojection error	重投影誤差	模型投影位置與已知真值位置的距離。
Baseline	基準方法	合理的簡單比較對象。
Ablation	消融實驗	拿掉某個模組，確認改進來源。
Domain shift	領域偏移	訓練與實際環境的場館、光線、鏡頭或球衣不同。

工具選擇矩陣

需求	優先考慮	優點	注意事項
快速建立 BBOX dataset	Roboflow / CVAT	UI 完整、協作與格式轉換方便	先寫標註規則，再開始大量標。
研究多種 detector	MMDetection	配置與模型庫豐富	設定複雜，部署需額外整理。
快速完成 YOLO baseline	Ultralytics	訓練、推論、track API 一致	版本更新快，需固定環境與參數。
幾何與相機處理	OpenCV	成熟、跨語言、函式完整	函式成功不代表點選擇正確。
靜態相機即時 MOT	ByteTrack	簡潔、低額外成本	無 appearance，長遮擋易換 ID。
移動相機或較多 ID swap	BoT-SORT	相機運動補償，可加 ReID	參數與成本較高。
單人近距離即時 pose	MediaPipe	輕量、容易整合	不等同高精度多人比賽 pose。
多人 pose 研究	MMPose / OpenPose 系列	模型選擇與研究彈性高	遠距、小人物與遮擋仍具挑戰。
實驗追蹤	W&B / MLflow	保存參數、曲線與 artifacts	需先定義一致命名與資料版本。

指標選擇速查

任務	建議指標	同時要看的失敗案例
Detection	Precision、Recall、AP / mAP、small-object recall	遠距球、重疊球員、裁判誤報。
Tracking	IDF1、HOTA、ID switches、coverage	遮擋、交叉、攝影機運動。
Homography	validation reprojection error、court error	視野邊緣、外插區域、相機位移。
Team classification	accuracy、macro F1、switch rate、reject rate	白色球衣、陰影、裁判、背面。
Pose	PCK、OKS、Keypoint AP、角度誤差	遮擋、出畫、身體旋轉。
Event timing	frame MAE、tolerance accuracy	出手型態不同、球短暫遺失。
End-to-end	任務成功率、最終分析誤差、使用者時間	上游錯誤如何傳到結論。

可延伸的專題方向

1. **球員位置分析**：比較 BBOX footpoint、腳踝 keypoint 與 segmentation footpoint 的 BEV 誤差。
2. **追蹤穩定度**：比較 SORT、ByteTrack 與 BoT-SORT 在固定與移動相機下的 IDF1。
3. **隊伍辨識**：比較 HSV、深度 embedding 與 track-level voting。
4. **球權估計**：以球與球員距離、遮擋與時序狀態推論 possession。
5. **投籃事件**：結合球軌跡、手腕距離與姿態訊號估計 release frame。
6. **空間分析**：以 BEV 路徑計算 spacing、paint occupancy 與 transition speed。
7. **姿態穩定**：比較單幀 Pose、移動平均與信心加權時序濾波對關節抖動與角度誤差的影響。
8. **不確定性**：讓系統輸出位置區間、事件置信度與自動拒答，而非強制給單一答案。

參考資料與進一步閱讀

- [1] Martin A. Fischler and Robert C. Bolles. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. *Communications of the ACM*, 1981.
- [2] Navneet Dalal and Bill Triggs. Histograms of Oriented Gradients for Human Detection. CVPR, 2005.
- [3] Ross Girshick et al. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. CVPR, 2014.
- [4] Shaoqing Ren et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS, 2015.
- [5] Joseph Redmon et al. You Only Look Once: Unified, Real-Time Object Detection. CVPR, 2016.
- [6] Nicolas Carion et al. End-to-End Object Detection with Transformers. ECCV, 2020.
- [7] Alex Bewley et al. Simple Online and Realtime Tracking. ICIIP, 2016.
- [8] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple Online and Realtime Tracking with a Deep Association Metric. ICIIP, 2017.
- [9] Yifu Zhang et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. ECCV, 2022.
- [10] Nir Aharon, Roy Orfaig, and Ben-Zion Bobrovsky. BoT-SORT: Robust Associations Multi-Pedestrian Tracking. arXiv:2206.14651, 2022.
- [11] Jonathon Luiten et al. HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking. International Journal of Computer Vision, 2021.
- [12] Zhe Cao et al. OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields. IEEE TPAMI, 2021.
- [13] Ke Sun et al. Deep High-Resolution Representation Learning for Human Pose Estimation. CVPR, 2019.
- [14] Valentin Bazarevsky et al. BlazePose: On-device Real-time Body Pose Tracking. arXiv:2006.10204, 2020.
- [15] Xiao Sun et al. Integral Human Pose Regression. ECCV, 2018.
- [16] Yanjie Li et al. SimCC: a Simple Coordinate Classification Perspective for Human Pose Estimation. ECCV, 2022.
- [17] Yufei Xu et al. ViTPose: Simple Vision Transformer Baselines for Human Pose Estimation. NeurIPS, 2022.
- [18] Tao Jiang et al. RTMPose: Real-Time Multi-Person Pose Estimation Based on MMPose. arXiv:2303.07399, 2023.
- [19] OpenCV Documentation. Basic Concepts of the Homography Explained with Code. https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html
- [20] Ultralytics Documentation. Multi-Object Tracking with BoT-SORT and ByteTrack. <https://docs.ultralytics.com/modes/track/>
- [21] Roboflow Documentation. Annotation Tools and Project Types. <https://docs.roboflow.com/annotate/annotation-tools>

最後提醒： 本講義旨在建立一套可持續擴充的分析框架：先定義問題與輸出，再選資料與方法；保留中間結果，逐層評估；最後用證據支持結論，並清楚說明不能做什麼。